Assignment 4

Aaron Bussche David Magaram Jason Kasdiran
Group 28

April 13, 2026

Introduction

Evolutionary algorithms (EA) are a population-based optimization method. They are based on natural selection and work by exploring new solution areas and refining good already found solutions, where candidate solutions are improved by mutating and crossover.

By investigating the traveling salesman problem and string search, this project explores how different parameter configurations influence convergence speed, population diversity, and algorithm performance.

Methods

This project will implement a genetic algorithm (GA) for string search and memetic algorithm (MA) for traveling salesman problem.

String search

A string search genetic algorithm was implemented and the used parameters are displayed in Table 1.

Table 1: String Search GA Parameters

Parameter	**Value**
L	16
Alphabet $\|\Sigma\|$	52
N	200
K	$\{2, 5\}$
p_c	1.0
μ	$\{0,\ 1/L,\ 3/L\}$ + sweep
Runs	10
$G_{\max}$	100

Traveling salesman problem

Results

B1

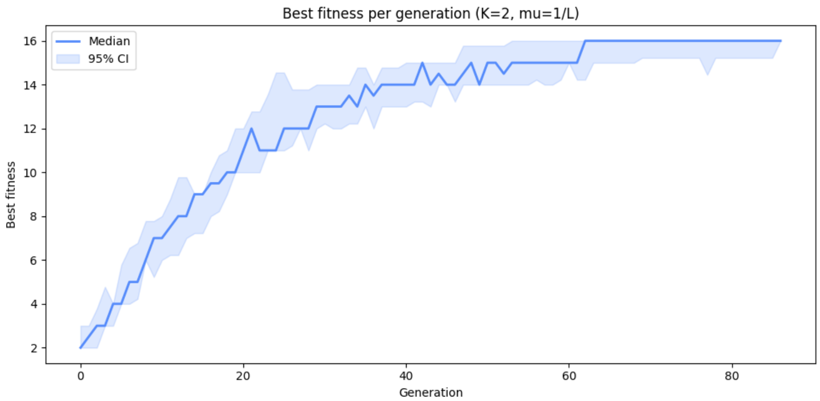

Figure 1: Results of running the GA 10 times with baseline values

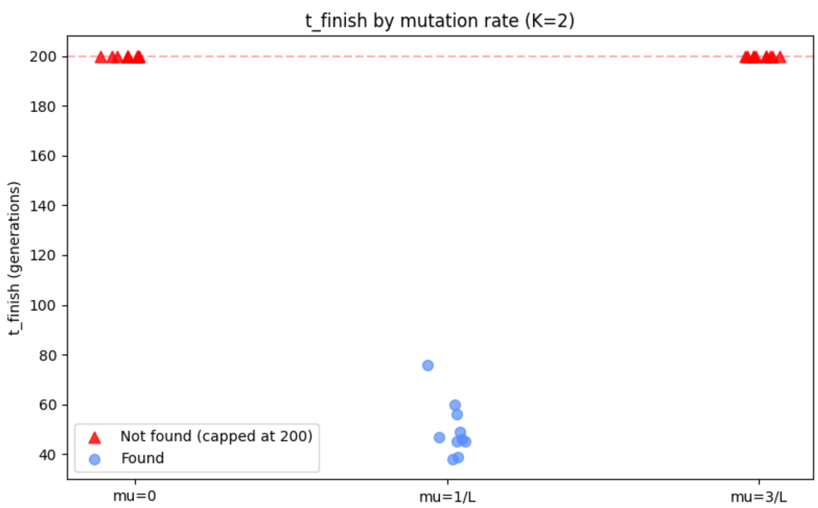

Figure 2: Three different mutation rates with K=2. Of these mutation rates, the GA manages to find the target only with mu=1/L before getting cut off at 200 Generations

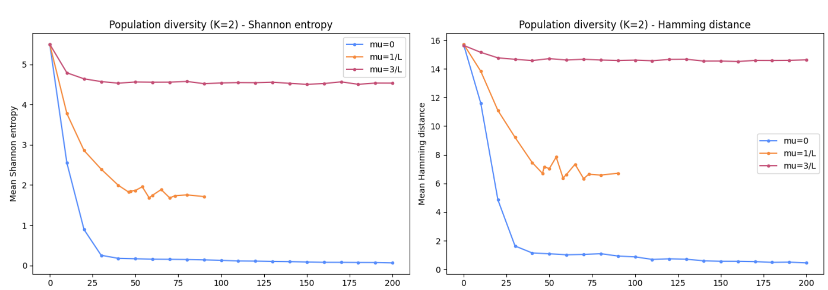

Figure 3: Measuring the diversity of the runs in two different ways, with Shannon entropy and Hamming distance. Both show that mu=0 leads to no diversity, mu=3/L leads to high diversity through randomness while mu=1/L manages to strike a middle ground (and reach the target).

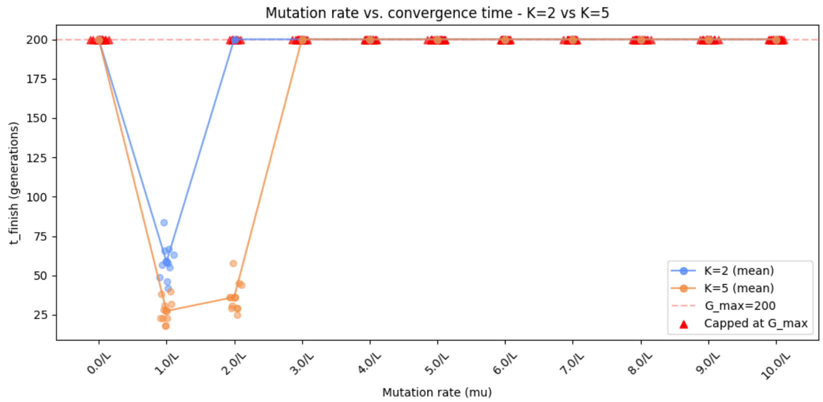

Figure 4: Higher tournament size increases selection pressure. We repeat the mu sweep to see how this interacts with mutation rate. K=5 leads to results faster & can also converge with mu=2/L, though mu=1/L remains optimal.

B2

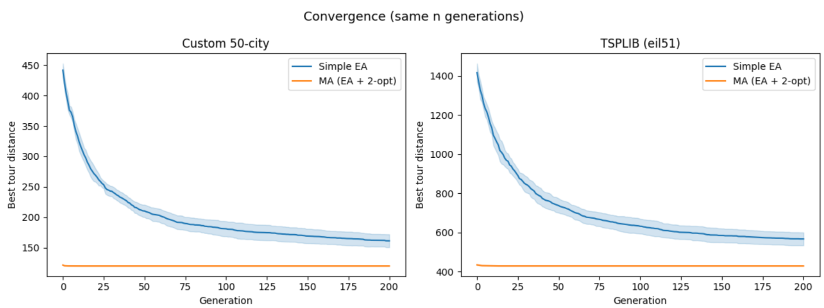

Figure 5: Both of the datasets show MA reaching better results in fewer generations, and in total.

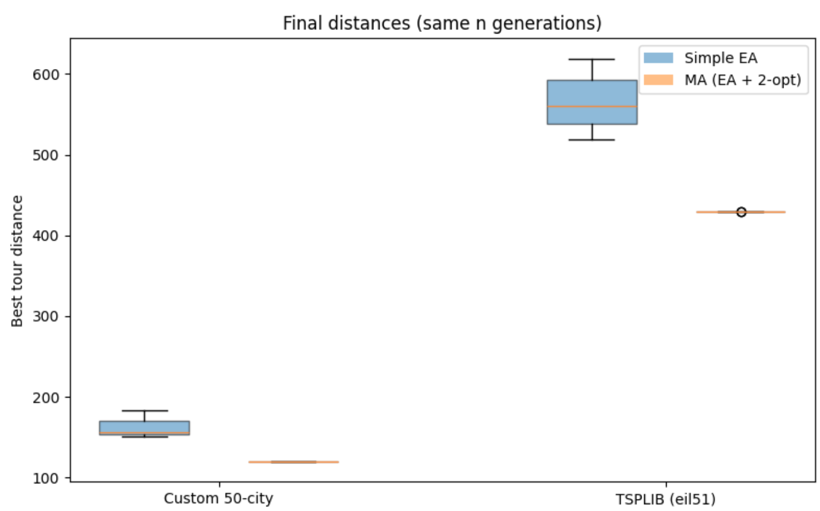

Figure 6: In this comparison, MA clearly beats EA in distance and consistency.

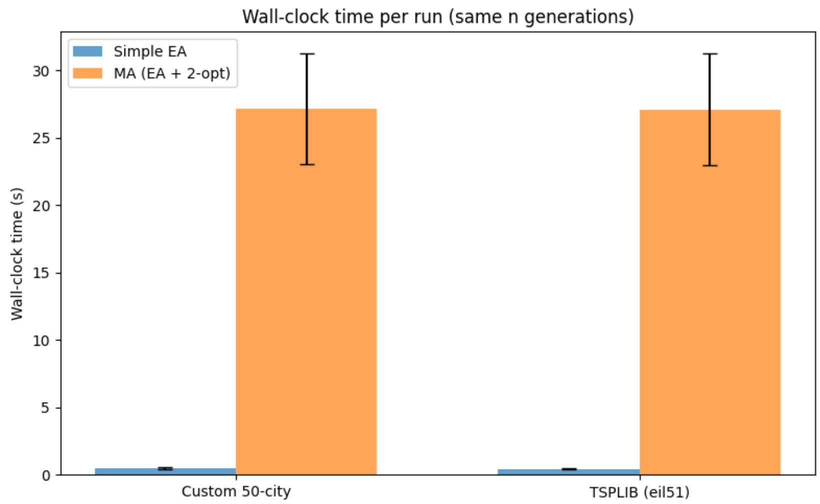

Figure 7: Per generation, MA takes much more compute & longer than EA. It is therefore not fair to compare them in this way.

We considered comparing them on number of operations, but the operations of MA are only in its neighbourhood, so comparing them directly with whole generations of EA would again be unfair. We therefore decided to compare wall-clock time that, while influenceable by all kinds of factors and therefore not overly scientific, provides a good baseline for real-use performance, especially if averaged over 10 runs each.

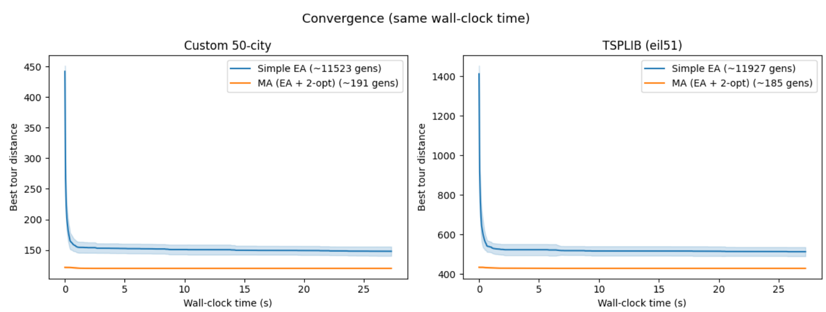

Figure 8: If compared instead on wall-clock time, MA still converges on better values, but EA's curve also flattens to near its best result in a comparable time as MA. While EA goes through more than 11-thousand generations in this time, MA only reaches around 190.

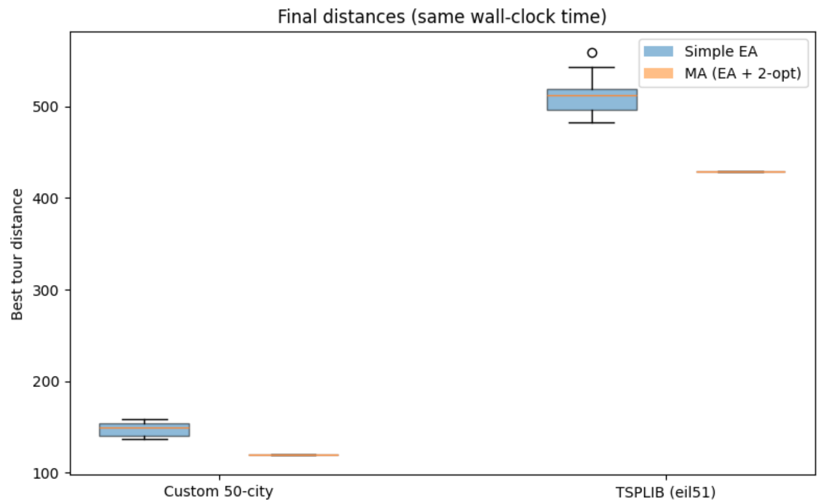

Figure 9: While the gap has shrunk considerably from comparing by number of generations, MA still provides better and more consistent results than EA.

Conclusion

Discussion

Appendix

Part A

A1

Add table and pie chart images.

What can you conclude about the effects of fitness scaling on selection pressure?

Adding constants raises all fitness all values by that amount. So it smooths out all the values. For example, when we compare f2 (which only squares X) and f4 (which squares X and adds 20), we can see that the probabilities of f4 are closer together. Probabilities are increased and reduced, so there is a much smaller gap between them, which also reduces selection pressure. Multiplying by a constant does not change the selection probabilities. If we compare f2 and f3, we can see that the probabilities stay the same even if we scale it. So scaling fitness does not have an effect on the selection pressure.

A2

Does the algorithm find the optimum? Yes. It is able to reach fitness 100, meaning all 100 bits were successfully set to 1.

What do you see? The version without selection is stuck at around 50 and never converges. The version with selection keeps steadily climbing, ultimately reaching 100.

Can you conclude anything on the difference between these two versions based on the above results? If yes, why? If not, what should you do to make a fair comparison? No. We would need multiple runs as EAs are stochastic and each run produces different results because of initialization and random mutation. To make it fair, we should run each version multiple times and use averages.

Is there a difference in performance? Yes. It is apparent that there is a difference in performance between the versions. Across all the runs, the version with selection always converges to fitness 100 while the version without selection is always near 50 through all the 1500 generations. The algorithm has no memory of good solutions and cannot improve.